

E-Contest

Shaastra 2026

Befunge - 93 Syntax Sheet

INTRODUCTION

Befunge is a two-dimensional esoteric programming language invented in 1993 by Chris Pressey with the goal of being as difficult to compile as possible.

Code is laid out on a two-dimensional grid of instructions, and execution can proceed in any direction of that grid.

A Befunge program is laid out on a two-dimensional playfield of fixed size. The playfield is an **80x25** rectangular grid of ASCII characters, each generally representing an instruction. **All the code must be written in these bounds or else you will get an error.**

Befunge programs mostly store data on a stack, but they can also read and write the contents of any cell in the playfield, given its coordinates, thus Befunge code can be **self-modifying**.

Characters are represented by their ASCII values within the stack, enabling operations like $A + D$, as they operate on the ASCII values of the characters.

Syntax

Basic syntax:

~	Get character from user and push its ascii value onto the stack.
&	Get integer from user and push it onto the stack
“	push each character's ASCII value all the way up to the next “
0 – 9	Push corresponding number onto the stack
.	Pop top of stack and output as integer. If the stack is empty, it pops and prints 0.
,	Pop top of stack and output as ASCII character
:	Duplicate top stack value. If the stack is empty, it considers the top of the stack as 0 and returns 2 0's onto the stack.
\	Swap top stack values. If there is only one element on the stack, it considers the second element as 0 and swaps them
\$	Pop top of stack and discard
@	End program

Example Code to print “Hello World”:

```
▼ Code
"!dlrow olleH",,,,,,,,,@
```

Explanation - “ starts pushing the ASCII values of characters onto the stack, in order, the second “ stops this process. Each comma(,) pops the top of the stack and outputs the characters. @ ends the programme.

Direction Pointers

The program is executed by the means of a program counter(PC). The direction in which the PC moves is decided by these direction pointers. Whenever the PC encounters these instructions, it changes its direction based on the instruction. Initially the PC starts at the top left corner and moves rightwards.

>	PC direction right
<	PC direction left
^	PC direction up
v	PC direction down
?	Random PC direction
#	Skips the next command in the current direction of the current PC

This is an infinite loop.

```
▼ Code
>v
^<
```

These direction pointers are also used along with if-else statements to implement loops in Befunge which will be explained in the further part of the document.

If Else Statements

—	Horizontal IF: pop a value; set direction to right if value=0, set to left otherwise
	Vertical IF: pop a value; set direction to down if value=0, set to up otherwise

Mathematical Operations

+	Addition: Pop two values a and b, then push the result of a+b
-	Subtraction: Pop two values a and b, then push the result of b-a
*	Multiplication: Pop two values a and b, then push the result of a*b
/	Integer division: Pop two values a and b, then push the result of b/a, rounded down. According to the specifications, if a is zero, ask the user what result they want.
%	Modulo: Pop two values a and b, then push the remainder of the integer division of b/a.
!	Logical NOT: Pop a value. If the value is zero, push 1; otherwise, push zero.
`	Greater than: Pop two values a and b, then push 1 if b>a, otherwise zero.

Example:

Factorial of n

```

▼ Code
&>:1-:v v *_$.@
^
_$>\: ^
  
```

This code takes an integer as an input and outputs the factorial of that number.

- First, '&' takes an input.
- Then we enter the first loop. In this loop, we duplicate(:) the number on the top of the stack, subtract 1 from it.
- Duplicate it again for the if statement. Whenever we encounter an if statement, the top of the stack is popped so we always need to be careful to duplicate the number on the stack so that we don't lose it.
- We exit from this loop whenever the top of the stack becomes 0. By the end of this loop, we will have all the numbers from n to 0 on the stack in order and 0 will be on the top of the stack.
- We now remove 0 from the stack using '\$'.
- Next, we enter our second loop. In this loop, we swap the top 2 elements of the stack, duplicate the top element for if statement, and then multiply the top two elements of the stack. By repeating this, eventually we will only have one element in the stack which will be n!.

- Now, when we swap the top two elements of the stack, as there is only one element, it considers the second element as 0 and brings it to the top. Now we duplicate 0 and when we pass it through the if statement, it exits the loop.
- Now the stack has 0 above n!. We now remove 0 using \$ and print n!

Let's take an example for $n = 3$

Operation	What is happening	Stack
&	Takes input as 3	[3]
>	PC moves to right	[3]
:	Duplicates and pushes	[3,3]
1	Pushes 1 on the stack	[1,3,3]
-	Subtracts the top two elements	[2,3]
:	Duplicates and pushes	[2,2,3]
v	PC moves down	[2,2,3]
—	If statement, pops the top element and checks if it is 0, it is not so PC moves left	[2,3]
^	PC moves up	[2,3]
>	PC moves to right	[2,3]
:	Duplicates and pushes	[2,2,3]
1	Pushes 1 on the stack	[1,2,2,3]
-	Subtracts the top two elements	[1,2,3]
:	Duplicates and pushes	[1,1,2,3]
v	PC moves down	[1,1,2,3]
—	If statement, pops the top element and checks if it is 0, it is not so PC moves left	[1,2,3]
^	PC moves up	[1,2,3]
>	PC moves to right	[1,2,3]
:	Duplicates and pushes	[1,1,2,3]
1	Pushes 1 on the stack	[1,1,1,2,3]
-	Subtracts the top two elements	[0,1,2,3]
:	Duplicates and pushes	[0,0,1,2,3]
v	PC moves down	[0,0,1,2,3]

—	If statement, pops the top element and checks if it is 0, PC moves right	[0,1,2,3]
\$	Pops the 0	[1,2,3]
\	Swaps the top two element	[2,1,3]
:	Duplicates and pushes	[2,2,1,3]
^	PC moves up	[2,2,1,3]
—	If statement, will go left at this point and pop 2	[2,1,3]
*	Multiples the top two numbers	[2,3]

We will just explain how the stack is moving now, because we hope this explanation makes the operations a bit clearer.

[2,3] -> [3,2] -> [3,3,2] -> [3,2] -> [6] -> [0,6] -> [0,0,6] -> [0,6] (loop ends here) -> [6]

At the end 6 is popped and printed and the program ends.

Special Instructions:

Apart from the above instructions, there exist 2 special instructions in Befunge; get(g) and put(p). These two are very powerful and must be used carefully. These are generally used to store data on the 2D playfield itself so that it becomes easier to access this data.

g	A "get" call (a way to retrieve data in storage). Pop two values y and x, then push the ASCII value of the character at that position in the program. If (x,y) is out of bounds, push 0
p	A "put" call (a way to store a value for later use). Pop three values y, x and v, then change the character at the position (x,y) in the program to the character with ASCII value v

The 'p' instruction can also be used to change the code while it is running but it is rarely done. It is usually used to store data in the empty parts of the code.

Example:

Factorial of n (using p and g instructions)

▼ Code

```
&:01p>01g1-:v
  ^ *p10:_$.@
```

This program also takes n as input and outputs its factorial. Notice that this code has only one loop as compared to the previous code without using p and g

- First, ' $\&$ ' takes an integer as input.
- Then, we duplicate the number and put it in the cell $(0,1)$. Notice that the first index is horizontal axis and the second one is vertical and the count starts from 0.
- Then we enter the loop. In this loop, we get the number m present at $(0,1)$, subtract 1 from it and duplicate it for the if statement. Then, we duplicate it again and place $m-1$ back at $(0,1)$. Then we multiply the top two numbers on the stack.
- We exit the loop once the number at $(0,1)$ becomes 1. After exiting the loop, the stack will contain 0, $n!$.
- Now we remove 0 using $\$$ and output $n!$.

Lets go for $n = 2$

Operation	What is happening	Stack
$\&$	Takes Input as 2	[2]
:	Duplicates and pushes	[2,2]
01	Pushes 0 and 1 to the stack	[1,0,2,2]
p	First pops two numbers $(0,1)$ and pops next number (2) and puts 2 at coordinate $(0,1)$	[2]
01g	Gets the number at $(0,1)$ coordinate	[2,2]
1-	Subtracts 1 from top	[1,2]
:	Duplicates and pushes	[1,1,2]
v	PC moves down	[1,1,2]
$-$	If statement, pops the first element and goes to left (not 0)	[1,2]
:	Duplicates and pushes	[1,1,2]
$p10$	Puts the top element 1 in $(0,1)$ (PC is moving leftwards)	[1,2]
$*$	Multiplies top two elements	[2]
$>$ $\wedge$	Changes PC direction, first to up then to right.	[2]
01g	Gets the number at $(0,1)$	[1,2]
1-	Subtracts 1 from the top element	[0,2]
:	Duplicates and pushes	[0,0,2]
v $-$	PC moves down, pops the topmost element of the stack (which is 0) and moves right	[0,0,2] $\rightarrow$ [0,2]
$\$$	Pops the top element	[2]

.	Pops the top element and prints it	[]
@	Ends the program	[]

Some more examples:

Hello world using '#' instruction

```
▼ Code
"!dlroW olleH">: #, _@
```

Hello world using loops

```
▼ Code
> v
v"Hello World!"<
>: v
^, _@
```

ASCII Table:

While writing programs, you might find it useful to know the ASCII values of certain characters.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]